

TitanTPP: Efficient History Correction for Next-Event Quantity Prediction

Anonymous Authors

Anonymous Institution

Abstract. Predicting demand events requires estimating both their timing and their quantity from an irregular history. We introduce TitanTPP, a compact event encoder with a bottleneck history correction between two causal attention blocks. The correction combines current and immediately preceding contextual states, while learned static banks supply additional context without online parameter updates. A common-input, common-head comparison covers six temporal point process encoders across four datasets and three seeds. On the development validation partitions, TitanTPP reduces mean quantity RMSE by 15.85% on Taxi and 34.39% on Intermittent relative to the strongest external encoder on each dataset. The improvement over the last-quantity baseline on Intermittent is smaller, at 2.04% in RMSE, and that baseline achieves lower MAE. RAF exhibits a small RMSE gain with an MAE trade-off, while Instacart favors external encoders on RMSE. Matched batch measurements against a Titans-MAC adapter indicate that the MAC adapter requires 5.58–5.74 times TitanTPP’s training-step compute time. TitanTPP reduces peak allocated training memory by approximately 75.5%, while its evaluation memory is higher. These results support compact history correction for next-event quantity prediction and identify the demand settings in which its benefits are strongest.

Keywords: Temporal point processes · Event quantity prediction · History encoding · Intermittent demand

1 Introduction

Demand planning concerns both the time of the next requirement and its size. When observations are sporadic, a history of event gaps and quantities provides a direct representation of those two quantities. Classical intermittent-demand methods separate demand occurrence from size [15]. Neural renewal models extend this event-based perspective through learned arrival and quantity distributions [5]. A useful event encoder must preserve predictive relationships in the history while remaining practical to train.

Attention combines information across events, and neural memory provides another route for historical context. Titans integrates attention with a memory module that adapts at test time [4]. For bounded event histories, a simpler question motivates our design: can a compact, feed-forward correction exploit recent-event relationships without online memory updates?

We propose TitanTPP, a causal encoder for next-event quantity and duration prediction. Two attention blocks surround a bottleneck residual that relates the current contextual state to its immediate observed predecessor. The first block encodes the history; the

correction exposes an explicit adjacent-state interaction; the second block integrates the adjusted representations. Learned persistent vectors and a static prototype bank complement this path. The correction adds 6,144 parameters, and all learned parameters remain fixed during inference.

Our evaluation separates quantity accuracy, duration likelihood, and computational cost. Under shared inputs, prediction heads, and training rules, TitanTPP improves quantity MAE and RMSE over six TPP encoders in every paired seed on Taxi and Intermittent. The gains are smaller on RAF and do not extend to the best Instacart RMSE. Simple quantity baselines further qualify the Intermittent result: repeating the last quantity achieves lower MAE, whereas TitanTPP achieves lower RMSE. Matched batch measurements provide a separate comparison with a Titans-MAC event adapter.

Our contributions are as follows.

1. We introduce a compact event-history correction architecture that combines adjacent contextual states between causal encoder blocks and retains static memory access without online updates.
2. We evaluate next-event quantity prediction across four datasets and three seeds, including six common-head TPP encoders, a correction-removal control, and two non-learned baselines. The results identify substantial gains over the learned comparators on Taxi and Intermittent and quantify their limits in other demand settings.
3. We characterize computational cost against a Titans-MAC adapter using matched input batches. The measurements distinguish lower training compute and memory from the higher allocation observed during evaluation.

2 Related Work

Event-history encoders. Hawkes processes model event dependence through self-excitation and mutual excitation [2]. Neural TPPs learn historical representations and conditional event distributions [20]. RMTTP summarizes history with a recurrent network [18], and the Neural Hawkes Process employs continuous-time LSTM states [12]. THP, SAHP, and AttNHP encode temporal dependencies through attention [25,21,8]. S2P2 combines continuous-time state evolution with event impulses and nonlinear transformations [32], drawing on HiPPO’s polynomial-projection framework [3]. We compare these encoder families through a common numerical-input and prediction-head interface.

Event distributions and quantity forecasting. Neural TPPs can parameterize an integrated intensity [27] or directly specify a conditional duration distribution [19]. Our duration head follows the latter approach and evaluates the probability mass of recorded integer durations. In demand forecasting, Croston-style methods and their extensions separate occurrence from size [15,6,23]. Deep renewal processes connect this decomposition to neural arrival-and-size models [5]; FlexTPP models mixed event attributes with distribution-specific heads [11]. These methods motivate the joint study of numerical quantities and event timing.

DeepAR learns probabilistic forecasts across related series [10]. PatchTST and iTransformer explore temporal patches and variate tokens, respectively [31,30]. Their calendar-horizon formulations differ from our next-event target. TitanTPP predicts the size and

gap of a positive-demand event; demand accumulated over a future calendar interval requires an additional event aggregation procedure.

Residual correction and memory. Residual learning [17] and bottleneck adapters [13] provide compact transformation paths. TitanTPP specializes this structure to adjacent event representations and trains it jointly with the encoder. Its zero output initialization initially preserves the base function, sharing a motivation with zero-initialized residual paths such as ReZero [28]. Memory networks retrieve learned context [24], while Titans updates a neural memory at test time [4]. Our design retains static retrieval and implements history correction through ordinary feed-forward operations. The efficiency comparison evaluates these complete computational paths.

3 Method

3.1 Observed Events and Causal Encoding

Let $\mathcal{H}_n = \{(t_i, q_i)\}_{i=1}^n$ denote an observed event history, with recorded inter-event durations Δ_i and quantities q_i . We predict the next quantity and recorded duration. Each training window contains an observed prefix and one target event. The target contributes to the loss only; both heads read the last observed state.

For padded windows, v_i indicates a valid row and w_i indicates a permitted observed row. With $o_i = v_i w_i$, the input embedding is

$$f_i = [\log(1 + \Delta_i), \log(1 + q_i)]^\top, \quad x_i = o_i(W_{\text{in}}f_i + b_{\text{in}} + p_i), \quad (1)$$

where p_i is a learned positional embedding and the hidden dimension is $d = 64$. Unobserved inputs are replaced by zero before the logarithm and projection. Durations follow each dataset’s recorded units, including the loader’s positive-gap convention for the first row.

The encoder contains two pre-normalized causal attention blocks [7,16]. Each block has four heads and a feed-forward width of 128. A learned bank $P^{(\ell)} \in \mathbb{R}^{16 \times d}$ supplies persistent keys and values in block ℓ . Suppressing head indices,

$$\begin{aligned} K &= [P^{(\ell)}; K_{\text{events}}], \quad V = [P^{(\ell)}; V_{\text{events}}], \\ A_i &= \text{softmax}(Q_i K^\top / \sqrt{d_h} + C_i) V, \end{aligned} \quad (2)$$

where $d_h = 16$ and the persistent bank denotes the corresponding head slice. The mask C_i permits all persistent vectors and only observed event keys at positions $j \leq i$. The same raw bank vectors serve as keys and values. Attention and GELU feed-forward transformations each have a residual path, with dropout 0.1. Unobserved query states are zeroed after each residual. Writing the blocks as $\mathcal{E}_1$ and $\mathcal{E}_2$, the first produces $H^{(1)} = \mathcal{E}_1(X)$.

3.2 Bottleneck History Correction

The intermediate correction explicitly relates the current contextual state to its immediate observed predecessor. Let $\pi(i)$ denote that predecessor, skipping padding, and let

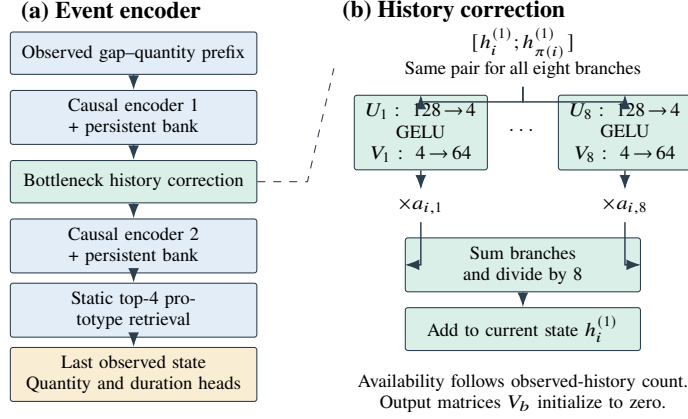


Fig. 1. TitanTPP and the expanded correction. The eight branches share an input pair but have separate parameters. The two persistent banks and final prototype bank are learned during training and fixed during inference. The next target event is withheld from the encoder.

c_i count observed rows since the latest withheld valid row, including row i . For eight thresholds $\tau = (1, 2, 4, 8, 16, 32, 64, 128)$, branch availability is

$$a_{i,b} = o_i \mathbf{1}\{c_i > \tau_b\}, \quad b = 1, \dots, 8. \quad (3)$$

Branch b contains bias-free matrices $U_b \in \mathbb{R}^{4 \times 2d}$ and $V_b \in \mathbb{R}^{d \times 4}$. Its output enters the residual

$$r_i = \frac{1}{8} \sum_{b=1}^8 a_{i,b} V_b \text{GELU}(U_b[h_i^{(1)}; h_{\pi(i)}^{(1)}]), \quad (4)$$

$$\tilde{h}_i^{(1)} = h_i^{(1)} + r_i, \quad H^{(2)} = \mathcal{E}_2(\tilde{H}^{(1)}). \quad (5)$$

GELU follows its exact error-function formulation [9]. All eight branches read the same immediate predecessor. The thresholds govern availability, not retrieval lag. The divisor remains eight when fewer branches are active. If no predecessor exists, all branches are unavailable and $r_i = 0$. Withheld valid rows reset correction eligibility; padding does not. This rule leaves the encoder’s causal attention history intact.

The correction adds $8(2dr + rd) = 6,144$ parameters for bottleneck width $r = 4$. Every output matrix V_b initializes to zero, so the correction initially preserves the base encoder’s function under the same stochastic state. Parameter construction preserves the caller’s random-number stream. Both inputs in Eq. (4) are contextual states, allowing the correction to depend on information preceding the adjacent event pair. Figure 1 summarizes the resulting computation.

3.3 Static Retrieval and Prediction Heads

A bank $M \in \mathbb{R}^{64 \times d}$ supplies static prototypes after the second block. At each observed position, cosine similarity selects four prototypes whose raw vectors enter an unweighted

residual:

$$\begin{aligned} S_i &= \text{TopK}_{j, k=4} \cos(h_i^{(2)}, m_j), \\ z_i &= o_i \left(h_i^{(2)} + \frac{1}{4} \sum_{j \in S_i} m_j \right). \end{aligned} \quad (6)$$

Normalization affects selection only. The bank ties keys and values and receives no on-line writes. Both heads read z_n , the last observed state.

The quantity head predicts a nonnegative point estimate through

$$\hat{y}_{n+1} = \text{softplus}(w_q^\top z_n + b_q), \quad \hat{q}_{n+1} = \exp(\hat{y}_{n+1}) - 1. \quad (7)$$

At initialization, $w_q = 0$ and b_q yields the training mean of $\log(1 + q)$. The duration head specifies a positive latent duration T :

$$\begin{aligned} \mu_n &= w_\mu^\top z_n + b_\mu, \quad \sigma_n = \text{softplus}(w_\sigma^\top z_n + b_\sigma) + 10^{-3}, \\ \log(T/s) \mid \mathcal{H}_n &\sim \mathcal{N}(\mu_n, \sigma_n^2). \end{aligned} \quad (8)$$

For recorded durations $D = \max(1, \text{round}(T))$, with optional upper coding at K , the probability mass is

$$p(D = d \mid \mathcal{H}_n) = \begin{cases} F_n(1.5), & d = 1, \\ 1 - F_n(K - 0.5), & d = K \text{ under upper coding}, \\ F_n(d + 0.5) - F_n(d - 0.5), & \text{otherwise}, \end{cases} \quad (9)$$

where F_n is the lognormal CDF. Instacart has $K = 30$; other datasets lack upper coding. The scales s are one hour, three weeks, seven days, and six months for Taxi, Intermittent, Instacart, and RAF, respectively. Stable float64 CDF or survival-function differences evaluate the log masses.

3.4 Training and Selection

The objective combines recorded-duration NLL and squared log-quantity error:

$$\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} \left[-\log p(D_{n+1} \mid \mathcal{H}_n) + (\hat{y}_{n+1} - \log(1 + q_{n+1}))^2 \right]. \quad (10)$$

Each window contributes one final target, and the quantity term has unit weight. AdamW [14] optimizes the objective with learning rate 10^{-3} , weight decay 0.01, gradient clipping at norm 1, and batch size 128. Training permits at most 300 epochs, at least 40 epochs, and patience 40. We select the earliest checkpoint attaining the strictly lowest finite validation RMSE on the raw quantity scale. MAE and duration NLL come from that same checkpoint.

At fixed width and branch count, the correction requires linear projection work in sequence length. The complete encoder retains the quadratic interactions of causal attention. This distinction motivates measuring the complete model rather than inferring its cost from the correction alone.

Table 1. Frozen train and validation targets. Maximum rows include the withheld target; lookbacks follow the dataset’s recorded units.

Dataset	Train	Validation	Max. rows	Lookback
Taxi	38,393	8,268	256	168 hours
Intermittent	393,824	86,285	256	520 weeks
Instacart	1,991,192	503,733	64	52 days
RAF	25,779	6,690	84	84 months

4 Experimental Setup

4.1 Data and Evaluation Scope

The four datasets encode different quantity-bearing events. Taxi counts pickups within a spatial cell and an active hour. Intermittent contains positive order quantities for site-part sequences. Instacart aggregates product rows for each user on the same recorded activity day, yielding a basket-size proxy rather than physical purchased units. RAF represents positive monthly spare-parts demand, with event gaps measured in months. Zero-demand calendar periods enter the gaps rather than separate quantity targets. Table 1 summarizes the target populations.

Frozen splits follow event order within each sequence, approximately 70/15/15. The comparisons in this draft concern development validation partitions, which also informed architecture choices. Taxi sequence selection and Intermittent filtering depend on full observed sequence lengths, defining retrospective cohorts. We distinguish these comparisons from an independent final evaluation in Sect. 7.

4.2 Comparators and Controls

The common-head panel contains RMTTP, NHP, THP, SAHP, S2P2, and AttNHP. Each receives the same gap-quantity history and target and connects to the same quantity and duration heads. All follow Eq. (10), the same optimizer settings, and the same checkpoint rule. This explicit interface follows the reproducibility motivation of TPP benchmarking efforts such as EasyTPP [26]. Implementations employ PyTorch [1].

S2P2 retains HiPPO initialization, complex diagonal evolution, backward zero-order-hold discretization, event impulses, relative time, and normalization/nonlinearity. The adaptation has width 64, two layers, and state size 16. Its affine-doubling recurrence was checked against the sequential equations; it incurs $O(L \log L)$ work rather than the original work-efficient scan. AttNHP retains query/context attention, sinusoidal time encoding, and tanh query residuals, with width 64, two layers, one attention head, time width 16, and no feed-forward sublayer. It queries the right limit of the last observed event, without the next target time. Numerical input projections and common heads replace the original categorical inputs and native outputs.

The B control retains the causal encoder and learned banks but removes the intermediate correction. It measures the effect of adding the history path, including its parameters. An exploratory normalization variant divides by the number of available branches

instead of eight. We retain a single representative TitanTPP architecture throughout the main comparison.

Two nonlearned baselines predict either the last observed quantity or the mean quantity in exactly the same observed input window. They require no training or seed averaging. Neither specifies a duration distribution, so duration NLL is undefined for them.

4.3 Metrics and Repeated Runs

We report raw quantity MAE and RMSE, and recorded-duration NLL from Eq. (9). RMSE selection assigns greater weight to large absolute quantity errors; MAE records average absolute error. Comparisons remain within each dataset because their quantity scales differ [22]. Duration NLL evaluates the probability assigned to the recorded observation through the logarithmic score [29].

Learned models have three seeds, 42, 52, and 62. Tables report their arithmetic means and sample standard deviations, and the analysis includes paired seeds. The common-head panel contains 84 completed runs across 28 dataset–model groups. Selected and last checkpoints, validation re-evaluations, and source identities are retained. All three metrics refer to the RMSE-selected checkpoint rather than separate metric-specific minima.

5 Results

5.1 Comparison with TPP Encoders

Table 2 reports the complete four-dataset comparison. On Taxi and Intermittent, TitanTPP achieves lower quantity MAE and RMSE than each external encoder in all 36 paired comparisons (two datasets, six comparators, three seeds). RMTTP is the strongest external model on the mean quantity metrics for both datasets. Relative to RMTTP, TitanTPP reduces MAE/RMSE by 10.13%/15.85% on Taxi and 21.72%/34.39% on Intermittent. Several encoders achieve better duration NLL, so quantity and timing rankings differ.

On RAF, TitanTPP’s mean RMSE of 33.9507 is close to S2P2’s 33.9947, a 0.13% difference. TitanTPP leads in two of three paired seeds. RMTTP achieves lower MAE, 9.1362 versus 9.2506, and NHP achieves lower time NLL. The result extends the evaluation to spare-parts demand while exposing a metric-dependent trade-off.

On Instacart, RMTTP has the lowest mean MAE, while S2P2 has the lowest RMSE and time NLL. TitanTPP’s MAE is 0.16% higher than RMTTP’s. Relative to S2P2 and AttNHP, TitanTPP reduces mean MAE by 0.18% and 0.11%, respectively, but increases mean RMSE by 0.52% and 0.31%. Both added comparators achieve lower RMSE in every paired seed. The improvement observed on Taxi and Intermittent therefore does not transfer uniformly to basket-size prediction.

5.2 Comparison with Simple Quantity Rules

Table 3 places the learned-model results beside two direct history summaries. TitanTPP improves both quantity metrics over both rules on Taxi, Instacart, and RAF. Intermittent

Table 2. Development validation results, mean $\pm$ sample standard deviation over three seeds. All metrics come from the same RMSE-selected checkpoint. Bold denotes the lowest mean within each dataset and metric in this common-head panel; the nonlearned baselines appear separately.

Model	Quantity MAE	Quantity RMSE	Time NLL
<i>Taxi</i>			
TitanTPP	25.6237 $\pm$ 0.8320	79.7111 $\pm$ 2.8260	1.0387 $\pm$ 0.2843
RMTTPP	28.5124 $\pm$ 0.6344	94.7283 $\pm$ 3.4006	1.5798 $\pm$ 0.9400
THP	32.4095 $\pm$ 0.8144	107.5977 $\pm$ 4.5040	0.6629 $\pm$ 0.0174
NHP	94.2694 $\pm$ 1.8752	318.8148 $\pm$ 5.3011	0.6516 $\pm$ 0.0021
SAHP	34.2260 $\pm$ 0.4539	120.5273 $\pm$ 1.9996	0.6848 $\pm$ 0.0146
S2P2	33.0231 $\pm$ 0.5569	106.3540 $\pm$ 1.9135	0.6658 $\pm$ 0.0242
AttNHP	35.9811 $\pm$ 1.6088	127.1290 $\pm$ 8.9919	0.6876 $\pm$ 0.0194
<i>Intermittent</i>			
TitanTPP	0.7005 $\pm$ 0.0541	1.6730 $\pm$ 0.0819	0.4971 $\pm$ 0.2095
RMTTPP	0.8949 $\pm$ 0.0494	2.5499 $\pm$ 0.1604	0.2748 $\pm$ 0.0047
THP	0.8993 $\pm$ 0.0661	2.7534 $\pm$ 0.1784	0.2911 $\pm$ 0.0045
NHP	4.5188 $\pm$ 0.1706	12.8755 $\pm$ 1.0405	0.4613 $\pm$ 0.0969
SAHP	1.8619 $\pm$ 0.0899	7.0298 $\pm$ 0.7673	0.3448 $\pm$ 0.0138
S2P2	0.9359 $\pm$ 0.0086	2.5758 $\pm$ 0.0766	0.2349 $\pm$ 0.0189
AttNHP	1.0413 $\pm$ 0.0938	3.0799 $\pm$ 0.2982	0.2946 $\pm$ 0.0569
<i>RAF</i>			
TitanTPP	9.2506 $\pm$ 0.0855	33.9507 $\pm$ 0.0588	3.5446 $\pm$ 0.0669
RMTTPP	9.1362 $\pm$ 0.0557	34.5503 $\pm$ 0.2506	3.5191 $\pm$ 0.1463
THP	9.3014 $\pm$ 0.1819	34.7445 $\pm$ 0.3204	3.3928 $\pm$ 0.0187
NHP	9.1793 $\pm$ 0.0882	36.2731 $\pm$ 0.3186	3.3889 $\pm$ 0.0556
SAHP	9.2332 $\pm$ 0.0209	35.4193 $\pm$ 0.2130	3.7415 $\pm$ 0.0745
S2P2	9.2730 $\pm$ 0.0517	33.9947 $\pm$ 0.1261	3.4187 $\pm$ 0.0284
AttNHP	9.1917 $\pm$ 0.0434	34.8766 $\pm$ 0.0676	3.9707 $\pm$ 0.0609
<i>Instacart</i>			
TitanTPP	3.9916 $\pm$ 0.0026	5.8827 $\pm$ 0.0048	2.8071 $\pm$ 0.0041
RMTTPP	3.9852 $\pm$ 0.0052	5.8775 $\pm$ 0.0213	2.8051 $\pm$ 0.0048
THP	4.0013 $\pm$ 0.0025	5.8582 $\pm$ 0.0253	2.8056 $\pm$ 0.0016
NHP	4.4923 $\pm$ 0.0304	6.7613 $\pm$ 0.0538	2.8132 $\pm$ 0.0080
SAHP	3.9979 $\pm$ 0.0099	5.8776 $\pm$ 0.0067	2.8018 $\pm$ 0.0013
S2P2	3.9987 $\pm$ 0.0063	5.8523 $\pm$ 0.0150	2.7970 $\pm$ 0.0014
AttNHP	3.9958 $\pm$ 0.0021	5.8648 $\pm$ 0.0194	2.8080 $\pm$ 0.0028

Table 3. Nonlearned quantity rules on the same validation targets and observed windows. TitanTPP entries are three-seed means; deterministic rules have one result and no seed standard deviation or duration NLL.

Dataset	Predictor	MAE	RMSE
Taxi	TitanTPP	25.6237	79.7111
	Last quantity	41.1951	134.3931
	Window mean	120.3771	368.7843
Intermittent	TitanTPP	0.7005	1.6730
	Last quantity	0.5647	1.7078
	Window mean	4.5537	13.7821
Instacart	TitanTPP	3.9916	5.8827
	Last quantity	4.9674	7.3473
	Window mean	4.0507	5.9030
RAF	TitanTPP	9.2506	33.9507
	Last quantity	11.3136	44.5742
	Window mean	10.8564	36.8499

is different. Repeating the last quantity achieves MAE 0.5647, below TitanTPP’s 0.7005, while TitanTPP reduces RMSE from 1.7078 to 1.6730, a 2.04% improvement. Thus the large gain over the TPP encoder panel does not imply a similarly large gain over a simple quantity predictor.

The Intermittent comparison also clarifies the role of the checkpoint criterion. RMSE and MAE express different error priorities. A predictor can reduce large squared errors while increasing average absolute error, and the reported comparison preserves that distinction instead of selecting a different checkpoint for each metric.

5.3 Effect of the History Correction

Removing the correction yields the B control in Table 4. Adding the 6,144-parameter path reduces mean MAE/RMSE by 10.89%/11.93% on Taxi and 7.68%/6.04% on Intermittent. On Instacart, MAE falls by about 0.04% and RMSE rises by about 0.05%. Mean time NLL increases on all three datasets. The correction benefits quantity accuracy most in the same two datasets that exhibit the largest gains over the external encoders.

An exploratory normalization variant replaces the fixed divisor in Eq. (4) with the number of available branches, with a minimum divisor of one. On RAF, its three-seed MAE is 9.2322 ± 0.0974 , RMSE is 33.8979 ± 0.0804 , and time NLL is 3.5631 ± 0.0978 . Relative to the representative model, MAE and RMSE fall by 0.20% and 0.16%, while time NLL rises. Each quantity metric improves in two of three seeds. Seed-42 comparisons improve quantity error on Taxi but worsen it on Intermittent. We treat this as structural sensitivity rather than choosing a different representative architecture for each dataset.

Table 4. Correction-removal control, mean $\pm$ sample standard deviation over three seeds. B retains the causal encoder and both types of learned bank, but omits the intermediate correction.

Model	MAE	RMSE	Time NLL
<i>Taxi</i>			
B	28.7545 ± 0.4526	90.5093 ± 0.5207	0.7247 ± 0.0668
TitanTPP	25.6237 ± 0.8320	79.7111 ± 2.8260	1.0387 ± 0.2843
<i>Intermittent</i>			
B	0.7588 ± 0.0658	1.7806 ± 0.0439	0.3846 ± 0.1034
TitanTPP	0.7005 ± 0.0541	1.6730 ± 0.0819	0.4971 ± 0.2095
<i>Instacart</i>			
B	3.9932 ± 0.0018	5.8796 ± 0.0166	2.8065 ± 0.0045
TitanTPP	3.9916 ± 0.0026	5.8827 ± 0.0048	2.8071 ± 0.0041

5.4 Where the Quantity Gains Occur

The quantity gains are concentrated in different parts of each dataset. Relative to the fixed external comparator with the best overall RMSE, TitanTPP improves all three upper-quantity bins on Taxi and Intermittent, but has higher RMSE in their two lower-quantity bins. RAF’s improvements at quantities up to 60 are largely offset by losses above 60. The complete stratum counts and errors appear in Appendix C.

Instacart shows the opposite aggregate trade-off. Targets at most 20 account for 89.37% of validation examples, and TitanTPP has lower mean MAE and RMSE in both corresponding bins. S2P2 performs better in all three bins above 20. Their contributions to the overall MSE difference are -1.5882 below or at 20 and $+1.9458$ above 20, explaining why a small MAE gain accompanies a higher RMSE (Fig. 4). S2P2’s lower RMSE in every nonempty history bin also shows that short histories alone do not isolate this disadvantage.

Train distributions provide context for these differences (Appendix B). Median usable histories are 129, 40, 3, and 5 events for Taxi, Intermittent, RAF, and Instacart. After within-series centering, adjacent log-quantity correlations are 0.749, 0.928, -0.105 , and 0.037, respectively. Stronger adjacent association may help history correction on Taxi and Intermittent; the stratum errors establish where gains occur, while this proposed explanation remains a hypothesis rather than an identified causal effect.

6 Computational Efficiency

6.1 Matched Measurement Protocol

We compare TitanTPP with a Titans-MAC event adapter on identical RTX 5080 input batches for Taxi and Intermittent. Each configuration has length 256 and batch size 128. Each of three repetitions contains five warm-up batches and 32 measured batches. GPU synchronization bounds compute timing; within-repetition medians are summarized by their mean and sample standard deviation across repetitions. Peak allocated and reserved

Table 5. Compute time in milliseconds per batch, mean $\pm$ sample standard deviation over three repetition medians. Ratios are paired within repetition.

Dataset	Path	TitanTPP	MAC	MAC/TitanTPP
Taxi	Train	31.447 ± 0.106	180.582 ± 6.056	5.74 ± 0.18
	Eval.	7.632 ± 0.009	64.827 ± 0.622	8.49 ± 0.08
Intermittent	Train	32.157 ± 0.023	179.472 ± 0.555	5.58 ± 0.01
	Eval.	7.909 ± 0.030	64.916 ± 0.645	8.21 ± 0.07

Peak allocated GPU memory (MiB)

Training



Evaluation (expanded scale)

**Fig. 2.** Mean peak allocated memory under matched batches. Taxi values are plotted; Intermittent has the same TitanTPP values and a MAC training allocation of 7,631.0 MiB instead of 7,631.3 MiB. Training and evaluation panels have different horizontal scales; numeric labels preserve the comparison.

memory counters are reset after warm-up. Evaluation includes target-loss computation. The runtime employs Python 3.12.13, PyTorch 2.11.0+cu130, CUDA 13.0, float32 model computation, disabled TF32, and deterministic execution settings. Training timing includes forward and backward computation, loss evaluation, gradient clipping, and the outer optimizer update; host-to-device transfer is excluded. The MAC adapter retains observed-history memory updates and a fixed-shape compiled scan. No new compiled graphs appear during measured batches, and model order reverses in the second repetition.

6.2 Time and Memory

Table 5 reports synchronized per-batch compute times. The MAC/TitanTPP ratio is 5.58–5.74 for training steps and 8.21–8.49 for evaluation batches. These are matched batch-computation measurements, not time-to-convergence estimates.

TitanTPP reduces peak allocated training memory from about 7,631 MiB to 1,870 MiB, approximately 75.5%. Evaluation reverses the allocation ordering: 389.2 MiB for TitanTPP versus 97.8 MiB for MAC, approximately 3.98 times as much. Figure 2 contrasts the two paths. Peak reserved training memory is 2,042.0 MiB for TitanTPP versus 7,670.0/7,646.0 MiB for MAC on Taxi/Intermittent. Evaluation reservations are 416.0 MiB versus 151.3/130.0 MiB. The configured length-256 models contain 96,003 and 122,310 trainable parameters, respectively.

These measurements support an efficient training path for the tested configuration. Their scope is the complete backbone implementation, so the time difference cannot be attributed solely to removing online memory updates. The higher evaluation allocation also motivates reporting memory separately for training and evaluation.

7 Discussion and Limitations

The results favor compact history correction for quantity prediction on Taxi and Intermittent, with smaller or reversed gains elsewhere. The last-quantity baseline’s lower Intermittent MAE is particularly informative: outperforming the learned encoder panel does not establish superiority over every useful demand predictor. Duration likelihood also follows a different ranking. The model is therefore best characterized through its quantity–timing–cost trade-offs across demand settings.

The reported partitions informed architecture development. An independent final evaluation still requires frozen model choices, an explicitly justified evaluation population, and uncertainty procedures. Prior access to legacy test artifacts prevents treating those data as automatically untouched. Three-seed means, sample standard deviations, and paired wins describe repeatability; they do not constitute a significance test.

The B comparison changes capacity as well as the correction path. It does not isolate the individual effects of bottleneck rank, insertion position, availability thresholds, or initialization. Parameter-matched current-state controls, all-available-branch controls, and an event-native Deep Renewal comparison are part of an ongoing extension and are outside the completed four-dataset panel reported here. Appendix A records the completed subset available at this draft’s observation cutoff, including unfavorable results. The current results do not establish that every design choice is necessary or optimal. Likewise, the static-retrieval removal comparison available for a different internal variant does not isolate that component’s necessity in the representative architecture.

The common-head panel compares numerical-input adaptations of six TPP encoders. Its objective and epoch cap do not equalize parameter counts, tuning opportunity, or runtime. Rankings therefore concern these adapters. The quantity head supplies a point estimate rather than a normalized quantity density, and Eq. (10) is a multi-task objective rather than a full joint marked-event likelihood.

The data also limit prospective and operational interpretations. Cohorts are retrospectively selected; Instacart is a basket-size proxy with upper-coded durations; and the local Intermittent source still requires upstream provenance and redistribution documentation. Inputs omit item identities, calendar covariates, and product composition. The task addresses the next recorded event, not inventory cost or accumulated calendar-horizon demand. Finally, the efficiency study covers short, warmed batch workloads on one GPU. Full-epoch time, time to matched accuracy, energy, and deployment latency require separate measurements.

8 Conclusion

TitanTPP adds a compact bottleneck correction of adjacent contextual states to a causal event encoder. Static learned banks complement this history path while parameters re-

main fixed during inference. A four-dataset, three-seed validation study identifies the strongest quantity gains over the common-head TPP panel on Taxi and Intermittent. RAF and Instacart expose smaller gains and metric-dependent reversals, and a last-quantity baseline narrows the Intermittent advantage to RMSE.

Matched batch measurements indicate lower training-step compute time and peak allocated training memory than a Titans-MAC adapter, alongside higher evaluation memory. Together, the results support compact history correction as a practical architecture for next-event quantity prediction. Completing the structural comparisons and an independent final evaluation will clarify how its benefits extend beyond the development partitions.

A Completed Subset of the Ongoing Extension

The following development results were available at the 2 October 2026, 07:14 KST observation cutoff. They extend the four-dataset comparison with parameter-matched structural controls and a native quantity model. The reported runs have completed training and selected/last validation re-evaluation; their archived checkpoints await verification. These preliminary results cover complete Taxi groups and two additional seed-42 conditions.

The current-only control removes the explicit predecessor input from the correction while retaining history in the causal encoder. Its eight 64–6–64 branches match the original correction’s 6,144 parameters and retain the availability thresholds and fixed divisor. The all-available control retains the original current–predecessor inputs and branch widths, but activates all eight branches wherever a predecessor exists, still dividing by eight. It differs from active-count normalization, which changes the divisor rather than availability.

The Deep Renewal comparison is an event-native adapter with raw gap–quantity inputs, two 64-unit LSTM layers, and shifted negative-binomial duration and quantity distributions. It retains its native two-NLL objective rather than the common quantity-regression loss. Input windows, targets, optimizer, maximum epochs, and RMSE checkpoint selection follow the shared protocol. It is a next-event adaptation, not a replication of calendar-horizon forecasting experiments.

Table 6. Completed Taxi groups from the ongoing extension, three-seed validation mean $\pm$ sample standard deviation. The original MLP is reused. The new conditions are preliminary pending archived-checkpoint verification.

Configuration	MAE	RMSE	Time NLL
Original MLP	25.624 $\pm$ 0.832	79.711 $\pm$ 2.826	1.039 $\pm$ 0.284
Current-only	28.813 $\pm$ 0.469	90.048 $\pm$ 1.513	0.882 $\pm$ 0.183
All-available	25.329 $\pm$ 0.295	78.121 $\pm$ 0.711	1.191 $\pm$ 0.354
Deep Renewal	91.475 $\pm$ 1.047	404.786 $\pm$ 2.978	0.790 $\pm$ 0.016

On Taxi, the original MLP improves mean MAE/RMSE over current-only by 11.07%/11.48%, with lower quantity errors in every paired seed. All-available improves mean MAE/RMSE over the original by 1.15%/1.99%, but increases mean time NLL; its MAE and RMSE gains occur in one and two of the three paired seeds, respectively. Thus the available evidence supports explicit predecessor correction without establishing that the original availability thresholds are optimal. Deep Renewal has larger quantity errors and lower time NLL in this setting. All three selected checkpoints are at epoch 300, so the comparison reflects the budget endpoint rather than a demonstrated convergence limit.

Two additional current-only seed-42 conditions are complete. On Intermittent, current-only MAE/RMSE/time NLL are 0.74519/1.77305/0.35229, versus 0.63848/1.57849/0.34683 for the original MLP at the same seed. On Instacart, the corresponding values are 3.99002/5.87224/2.80332 versus 3.99150/5.87873/2.80322. Instacart’s small quantity difference favors current-only, providing a counterexample to a universal benefit from the explicit predecessor input. The all-available Intermittent and Instacart seed-42 fits are in progress, and RAF extension fits have not started. No three-seed conclusion is drawn for these incomplete groups.

B Data Definitions and Training Distributions

Taxi events are pickup counts in active spatial-cell hours; Intermittent events are positive site-part order quantities; RAF events are positive monthly spare-parts demands. Instacart events aggregate product rows for a user on the same recorded activity day. Its quantity is a basket-size proxy, not a count of physical units. In these event sequences, empty calendar periods contribute to the recorded gap rather than separate zero-quantity targets. Gaps retain their dataset-specific units (Table 7).

We describe each frozen training population using the same definitions. Quantity quantiles include all train event rows. Gap and usable-history statistics describe the canonical prediction targets, which exclude the first event of each series. For a target, usable history is the number of observed predecessor events retained by the dataset’s recorded-time window and sequence-length cap. It excludes the target and padding. Thus it differs from both elapsed calendar time and the total number of events in a series. Train quantities and histories characterize available learning evidence; the validation counts in Appendix C describe a different population later in each sequence.

For adjacent quantities, let $x_{e,j} = \log(1 + q_{e,j-1})$ and $y_{e,j} = \log(1 + q_{e,j})$ for train pairs in series e . We compute

$$r_{\text{within}} = \text{Corr}(x_{e,j} - \bar{x}_e, y_{e,j} - \bar{y}_e), \quad (11)$$

pooling the centered pairs across series. The two means are computed separately over each series’ predecessor and successor members. This pair-weighted statistic removes differences in average scale between series; it is not an average of series-specific correlation coefficients.

Taxi and Intermittent retain substantially longer histories and stronger within-series adjacent association than the other datasets. RAF’s median usable train history is three events, and 93.58% of targets have at most seven; the corresponding Instacart values are five and 72.13%. RAF’s centered correlation is -0.105 , compared with Instacart’s

Table 7. Train-only event characteristics. Quantity summaries use all train event rows; gap, usable-history, and adjacent-pair summaries use train targets after excluding each series’ first event. Quantiles use the nearest observed value. Instacart gaps are top-coded at 30 days.

Characteristic	Taxi	Intermittent	RAF	Instacart
Time unit	Hour	Week	Month	Day
Train series	131	5,000	5,000	206,209
Train event rows	38,524	398,824	30,779	2,197,401
Train targets	38,393	393,824	25,779	1,991,192
Quantity: median	7	2	2	8
Quantity: 95th percentile	1,562	46	60	25
Quantity: 99th percentile	3,449	187	200	35
Quantity: maximum	6,322	477	2,062	175
Recorded gap: median	1	3	6	7
Recorded gap: 95th percentile	5	5	23	30
Usable history: median	129	40	3	5
Usable history: 95th percentile	168	144	8	14
History ≤ 7 events (%)	2.41	8.89	93.58	72.13
Within-series log correlation	0.749	0.928	-0.105	0.037
Equal adjacent quantities (%)	20.33	61.99	33.30	10.24

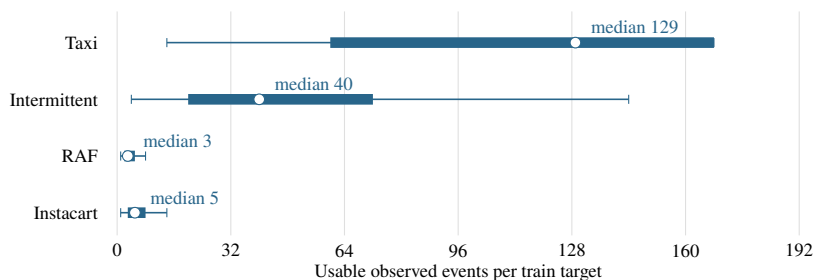


Fig. 3. Train-history distributions under the frozen input windows. Thin lines span the 5th–95th percentiles, thick lines the 25th–75th percentiles, and circles mark medians. These are event-weighted train summaries, not validation performance curves.

0.037. The RAF estimate summarizes very short sequences after mean removal, so it should not be read as a stable negative dependence law. These distributions make history correction’s dataset dependence plausible, while the observed error comparisons below establish where its gains and losses occur.

C Validation Errors by Quantity and Usable History

C.1 Partitions and Aggregation

The analysis uses the completed selected-checkpoint validation records for all seven models in Table 2: four datasets and three seeds, 84 runs in total. Every stratum uses the same earliest minimum-RMSE checkpoint as the overall results. The tables focus on TitanTPP and the external model with the lowest overall mean validation RMSE on each dataset: RMTTP on Taxi and Intermittent, and S2P2 on RAF and Instacart. This comparator is held fixed across strata, including strata where another model performs better.

Quantity boundaries are the frozen train-derived 50th, 90th, 95th, and 99th percentiles: (7, 686, 1562, 3449) for Taxi, (2, 31, 46, 187) for Intermittent, (2, 30, 60, 200) for RAF, and (8, 20, 25, 35) for Instacart. Upper boundaries belong to the lower bin. History boundaries reuse the stored partitions: (64, 128) observed events for Taxi, Intermittent, and RAF, and (1, 3, 7, 15, 31) for Instacart. The finer Instacart partition resolves its shorter histories. These dataset-specific bins support within-dataset diagnosis; they are not identical history-length groups across all four datasets. RAF’s validation targets all fall in the first stored history bin, so that partition cannot resolve its short-history differences. We retain the empty cells in Table 9.

Within each seed, MAE averages absolute error over targets and RMSE is the square root of mean squared error. Tables 8 and 9 report arithmetic means of those metrics over seeds 42, 52, and 62. Each N counts targets once per seed, not three times; the target populations are identical across models and seeds. Duration NLL in the overall comparison also uses the same selected checkpoints.

C.2 Contributions to the Aggregate Difference

For stratum b with N_b of the N validation targets, define the contribution to the MSE difference as

$$C_b = \frac{1}{3N} \sum_{s \in \{42, 52, 62\}} (\text{SSE}_{\text{TitanTPP}, s, b} - \text{SSE}_{\text{external}, s, b}). \quad (12)$$

Then $\sum_b C_b$ equals the mean of the three seeds’ overall MSE differences. It is not the difference of squared mean RMSEs. For MAE, the analogous contribution is N_b/N times the mean within-stratum MAE difference. Figure 4 shows both decompositions for Instacart.

On Taxi, the bins at or below 686 contribute +200.886 squared quantity units, whereas the bins above 686 contribute −2822.868, yielding a net −2621.982 relative to RMTTP. Intermittent has the same direction of concentration: the bins at or below 31 contribute +0.05969 and those above 31 contribute −3.77546. These gains are concentrated rather than uniform: RMTTP has lower RMSE in the two lower-quantity bins of both datasets. Their history partitions also differ. Taxi’s gain is concentrated above 128 observed events, while Intermittent improves in the first two history bins and worsens above 128. Longer history alone therefore does not order the benefits consistently.

RAF exhibits a near cancellation against S2P2. The bins at or below 60 contribute −31.463 squared quantity units and those above 60 contribute +28.463, leaving −3.000 overall. This explains why appreciable stratum-level differences coexist with a small aggregate RMSE improvement. The comparison is specific to S2P2; RMTTP retains the lowest overall RAF MAE in Table 2.

For Instacart, the bins at or below 20 contribute −1.58816 squared quantity units and those above 20 contribute +1.94581, leaving an MSE increase of 0.35765. The corresponding MAE difference is −0.00712. Thus the low-quantity majority yields a small MAE advantage, while the larger high-quantity errors reverse the RMSE ranking. S2P2 also has lower mean RMSE in every nonempty stored history bin. The observed disadvantage cannot therefore be assigned solely to a single short-history group.

Table 8. Quantity-stratum validation errors at the selected checkpoints. Entries are arithmetic means over seeds 42, 52, and 62; N is the number of targets per seed. The external comparator is fixed per dataset by the overall RMSE in Table 2, not selected within each bin. Empty strata have no estimated error.

Stratum	N	TitanTPP		External comparator	
		MAE	RMSE	MAE	RMSE
<i>Taxi vs. RMTPP</i>					
$q \leq 7$	4,364	2.3003	9.8889	2.1501	7.3292
$7 < q \leq 686$	3,138	23.4775	50.8082	21.9881	46.0420
$686 < q \leq 1562$	386	114.2264	160.7697	139.5607	201.5555
$1562 < q \leq 3449$	301	197.8383	271.7508	242.0791	320.5319
$q > 3449$	79	310.1894	384.9824	387.6200	509.8502
<i>Intermittent vs. RMTPP</i>					
$q \leq 2$	45,036	0.1487	0.2621	0.0731	0.1535
$2 < q \leq 31$	29,579	0.8965	1.4835	1.0205	1.4419
$31 < q \leq 46$	7,124	1.4464	2.2055	1.7433	2.4149
$46 < q \leq 187$	3,405	2.9671	4.2619	5.2337	6.6144
$q > 187$	1,141	5.9795	8.1891	11.8304	16.3628
<i>RAF vs. S2P2</i>					
$q \leq 2$	3,626	1.9835	6.3967	1.9552	7.1732
$2 < q \leq 30$	2,555	7.4744	14.3371	7.6838	15.9466
$30 < q \leq 60$	235	25.5628	30.9633	28.4258	33.9447
$60 < q \leq 200$	224	69.3894	81.9318	65.8432	77.9821
$q > 200$	50	280.9391	325.8366	277.7154	324.3119
<i>Instacart vs. S2P2</i>					
$q \leq 8$	247,651	2.7184	3.8081	2.8722	4.0783
$8 < q \leq 20$	202,534	3.6871	4.6741	3.7605	4.8145
$20 < q \leq 25$	27,322	7.9877	9.0866	7.4211	8.6749
$25 < q \leq 35$	20,190	11.8469	13.1561	10.6341	12.2964
$q > 35$	6,036	22.0811	24.6419	20.5236	23.4686

Table 9. Usable-history-stratum validation errors at the selected checkpoints. Entries are arithmetic means over seeds 42, 52, and 62; N is the number of targets per seed. The external comparator is fixed per dataset by the overall RMSE in Table 2, not selected within each bin. Empty strata have no estimated error.

Stratum	N	TitanTPP		External comparator	
		MAE	RMSE	MAE	RMSE
<i>Taxi vs. RMTPP</i>					
$h \leq 64$	1,241	0.5174	0.8503	0.5107	0.8422
$64 < h \leq 128$	1,475	1.2124	2.5664	1.2119	2.6314
$h > 128$	5,552	37.7208	97.2636	42.0243	115.5907
<i>Intermittent vs. RMTPP</i>					
$h \leq 64$	25,088	0.8210	1.3842	1.0308	2.0659
$64 < h \leq 128$	28,456	1.1389	2.5081	1.6594	3.9493
$h > 128$	32,741	0.2272	0.6452	0.1263	0.5260
<i>RAF vs. S2P2</i>					
$h \leq 64$	6,690	9.2506	33.9507	9.2730	33.9947
$64 < h \leq 128$	0	—	—	—	—
$h > 128$	0	—	—	—	—
<i>Instacart vs. S2P2</i>					
$h \leq 1$	261	7.1888	10.6058	7.1550	10.5516
$1 < h \leq 3$	130,279	4.0556	6.0319	4.0574	5.9661
$3 < h \leq 7$	205,041	3.9482	5.7901	3.9612	5.7591
$7 < h \leq 15$	140,970	4.0272	5.8757	4.0354	5.8739
$15 < h \leq 31$	26,298	3.7877	5.7835	3.7780	5.7828
$h > 31$	884	4.0527	6.9197	3.8184	6.9017

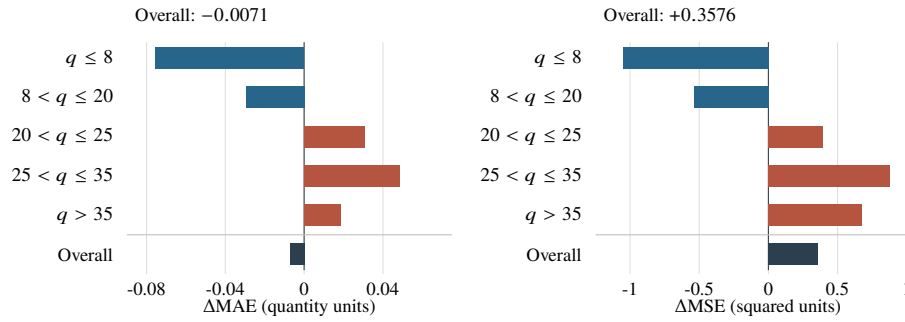


Fig. 4. Instacart quantity-stratum contributions to the overall error difference, TitanTPP minus S2P2. Each bar includes its stratum’s target share; negative values favor TitanTPP. The five bars sum to the overall difference in each panel. The panels have different units and horizontal scales.

References

1. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al.: PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems* 32 (2019).
2. Hawkes, A. G.: Spectra of some self-exciting and mutually exciting point processes. *Biometrika* **58**(1), 83–90 (1971). <https://doi.org/10.1093/biomet/58.1.83>
3. Gu, A., Dao, T., Ermon, S., Rudra, A., Ré, C.: HiPPO: Recurrent Memory with Optimal Polynomial Projections. *Advances in Neural Information Processing Systems* 33 (2020).
4. Behrouz, A., Zhong, P., Mirrokni, V.: Titans: Learning to Memorize at Test Time. *Advances in Neural Information Processing Systems* 38, 125925–125962 (2025). <https://doi.org/10.52202/085713-3786>
5. Türkmen, A. C., Januschowski, T., Wang, Y., Cemgil, A. T.: Forecasting intermittent and sparse time series: A unified probabilistic framework via deep renewal processes. *PLOS ONE* **16**(11), e0259764 (2021). <https://doi.org/10.1371/journal.pone.0259764>
6. Syntetos, A. A., Boylan, J. E.: The accuracy of intermittent demand estimates. *International Journal of Forecasting* **21**(2), 303–314 (2005). <https://doi.org/10.1016/j.ijforecast.2004.10.001>
7. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al.: Attention Is All You Need. *Advances in Neural Information Processing Systems* 30 (2017).
8. Yang, C., Mei, H., Eisner, J.: Transformer Embeddings of Irregularly Spaced Events and Their Participants. *International Conference on Learning Representations* (2022).
9. Hendrycks, D., Gimpel, K.: Gaussian Error Linear Units (GELUs). *arXiv:1606.08415* (2016).
10. Salinas, D., Flunkert, V., Gasthaus, J., Januschowski, T.: DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting* **36**(3), 1181–1191 (2020). <https://doi.org/10.1016/j.ijforecast.2019.07.001>
11. Draxler, F., Meng, Y., Nelson, K., Laskowski, L., Yang, Y., Karaletsos, T., et al.: Transformers for Mixed-type Event Sequences. *Advances in Neural Information Processing Systems* 38, 127441–127473 (2025). <https://doi.org/10.52202/085713-3833>
12. Mei, H., Eisner, J.: The Neural Hawkes Process: A Neurally Self-Modulating Multivariate Point Process. *Advances in Neural Information Processing Systems* 30 (2017).
13. Houshy, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., et al.: Parameter-Efficient Transfer Learning for NLP. *Proceedings of the 36th International Conference on Machine Learning* **97**, 2790–2799 (2019).

14. Loshchilov, I., Hutter, F.: Decoupled Weight Decay Regularization. *International Conference on Learning Representations* (2019).
15. Croston, J. D.: Forecasting and Stock Control for Intermittent Demands. *Journal of the Operational Research Society* **23**(3), 289–303 (1972). <https://doi.org/10.1057/jors.1972.50>
16. Ba, J. L., Kiros, J. R., Hinton, G. E.: Layer Normalization. *arXiv:1607.06450* (2016).
17. He, K., Zhang, X., Ren, S., Sun, J.: Deep Residual Learning for Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778 (2016). <https://doi.org/10.1109/CVPR.2016.90>
18. Du, N., Dai, H., Trivedi, R., Upadhyay, U., Gomez-Rodriguez, M., Song, L.: Recurrent Marked Temporal Point Processes: Embedding Event History to Vector. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1555–1564 (2016). <https://doi.org/10.1145/2939672.2939875>
19. Shchur, O., Biloš, M., Günnemann, S.: Intensity-Free Learning of Temporal Point Processes. *International Conference on Learning Representations* (2020).
20. Shchur, O., Türkmen, A. C., Januschowski, T., Günnemann, S.: Neural Temporal Point Processes: A Review. *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, 4585–4593 (2021). <https://doi.org/10.24963/ijcai.2021/623>
21. Zhang, Q., Lipani, A., Kirnap, O., Yilmaz, E.: Self-Attentive Hawkes Process. *Proceedings of the 37th International Conference on Machine Learning* **119**, 11183–11193 (2020).
22. Hyndman, R. J., Koehler, A. B.: Another look at measures of forecast accuracy. *International Journal of Forecasting* **22**(4), 679–688 (2006). <https://doi.org/10.1016/j.ijforecast.2006.03.001>
23. Teunter, R. H., Syntetos, A. A., Babai, M. Z.: Intermittent demand: Linking forecasting to inventory obsolescence. *European Journal of Operational Research* **214**(3), 606–615 (2011). <https://doi.org/10.1016/j.ejor.2011.05.018>
24. Sukhbaatar, S., Szlam, A., Weston, J., Fergus, R.: End-To-End Memory Networks. *Advances in Neural Information Processing Systems* **28** (2015).
25. Zuo, S., Jiang, H., Li, Z., Zhao, T., Zha, H.: Transformer Hawkes Process. *Proceedings of the 37th International Conference on Machine Learning* **119**, 11692–11702 (2020).
26. Xue, S., Shi, X., Chu, Z., Wang, Y., Hao, H., Zhou, F., et al.: EasyTPP: Towards Open Benchmarking Temporal Point Processes. *International Conference on Learning Representations* (2024).
27. Omi, T., Ueda, N., Aihara, K.: Fully Neural Network based Model for General Temporal Point Processes. *Advances in Neural Information Processing Systems* **32** (2019).
28. Bachlechner, T., Majumder, B. P., Mao, H., Cottrell, G., McAuley, J.: ReZero is all you need: fast convergence at large depth. *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence* **161**, 1352–1361 (2021).
29. Gneiting, T., Raftery, A. E.: Strictly Proper Scoring Rules, Prediction, and Estimation. *Journal of the American Statistical Association* **102**(477), 359–378 (2007). <https://doi.org/10.1198/016214506000001437>
30. Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., et al.: iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. *International Conference on Learning Representations* (2024).
31. Nie, Y., Nguyen, N. H., Sinthong, P., Kalagnanam, J.: A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. *International Conference on Learning Representations* (2023).
32. Chang, Y., Boyd, A., Xiao, C., Kass-Hout, T., Bhatia, P., Smyth, P., et al.: Deep Continuous-Time State-Space Models for Marked Event Sequences. *Advances in Neural Information Processing Systems* **38**, 180555–180596 (2025). <https://doi.org/10.52202/085713-5431>